

Hadoop Word Count Using MapReduce

1. Objective

To create a text file, store it in HDFS, execute the Word Count MapReduce program using Hadoop, and view the word frequency results.

2. Procedure

Step 1: Create a New Text File

```
cat > tech.txt << 'EOF'
```

Artificial Intelligence is transforming industries across the world.

Machine Learning enables computers to learn from data.

Big Data technologies help organizations process massive datasets.

Hadoop provides distributed storage and processing capabilities.

Spark enables fast data analytics and machine learning workflows.

EOF

Step 2: Verify the File Contents

```
cat tech.txt
```

Step 3: Upload the File to HDFS

```
hdfs dfs -put tech.txt /demo
```

Step 4: Execute the Word Count Program

```
hadoop jar $HADOOP_HOME/share/hadoop/mapreduce/hadoop-mapreduce-examples-*.jar  
wordcount /demo /output
```

Step 5: Check the Output Directory

```
hdfs dfs -ls /output
```

Step 6: View the Word Count Results

```
hdfs dfs -cat /output/part-r-00000
```

3. Source Code

Input File (tech.txt)

Artificial Intelligence is transforming industries across the world.

Machine Learning enables computers to learn from data.

Big Data technologies help organizations process massive datasets.

Hadoop provides distributed storage and processing capabilities.

Spark enables fast data analytics and machine learning workflows.

Commands Used

```
cat > tech.txt << 'EOF'
```

Artificial Intelligence is transforming industries across the world.

Machine Learning enables computers to learn from data.

Big Data technologies help organizations process massive datasets.

Hadoop provides distributed storage and processing capabilities.

Spark enables fast data analytics and machine learning workflows.

EOF

```
cat tech.txt
```

```
hdfs dfs -put tech.txt /demo
```

```
hadoop jar $HADOOP_HOME/share/hadoop/mapreduce/hadoop-mapreduce-examples-*.jar  
wordcount /demo /output
```

```
hdfs dfs -ls /output
```

```
hdfs dfs -cat /output/part-r-00000
```

4. Screenshots

Creating the Input File

```
vikas@Vikash:/mnt/c/Users/Vikas$ cat > tech.txt << 'EOF'  
Artificial Intelligence is transforming industries across the world.  
Machine Learning enables computers to learn from data.  
Big Data technologies help organizations process massive datasets.  
Hadoop provides distributed storage and processing capabilities.  
Spark enables fast data analytics and machine learning workflows.  
EOF
```

Verifying the File

```
vikas@Vikash:/mnt/c/Users/Vikas$ cat tech.txt  
Artificial Intelligence is transforming industries across the world.  
Machine Learning enables computers to learn from data.  
Big Data technologies help organizations process massive datasets.  
Hadoop provides distributed storage and processing capabilities.  
Spark enables fast data analytics and machine learning workflows.  
vikas@Vikash:/mnt/c/Users/Vikas$ |
```

Running Word Count

```

vikas@vikash:/mnt/c/Users/Vikas$ hadoop jar $HADOOP_HOME/share/hadoop/mapreduce/hadoop-mapreduce-examples-4.jar wordcount /demo /output
2026-06-25 05:50:15,259 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
2026-06-25 05:50:16,909 INFO client.DefaultHadoopFailoverProxyProvider: Connecting to ResourceManager at /0.0.0.0:8032
2026-06-25 05:50:17,062 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /tmp/hadoop-yarn/staging/vikas/.staging/job_1782357914992_0005
2026-06-25 05:50:17,915 INFO input.FileInputFormat: Total input files to process : 1
2026-06-25 05:50:18,443 INFO mapreduce.JobSubmitter: number of splits:1
2026-06-25 05:50:18,906 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1782357914992_0005
2026-06-25 05:50:18,906 INFO mapreduce.JobSubmitter: Executing with tokens: []
2026-06-25 05:50:19,220 INFO conf.Configuration: resource-types.xml not found
2026-06-25 05:50:19,229 INFO resource.ResourceUtils: Unable to find 'resource-types.xml'.
2026-06-25 05:50:19,347 INFO impl.YarnClientImpl: Submitted application application_1782357914992_0005
2026-06-25 05:50:19,413 INFO mapreduce.Job: The url to track the job: http://vikash.localdomain:8088/proxy/application_1782357914992_0005/
2026-06-25 05:50:19,414 INFO mapreduce.Job: Running job: job_1782357914992_0005
2026-06-25 05:50:29,682 INFO mapreduce.Job: Job job_1782357914992_0005 running in uber mode : false
2026-06-25 05:50:29,706 INFO mapreduce.Job: map 0% reduce 0%
2026-06-25 05:50:35,840 INFO mapreduce.Job: map 100% reduce 0%
2026-06-25 05:50:40,891 INFO mapreduce.Job: map 100% reduce 100%
2026-06-25 05:50:42,924 INFO mapreduce.Job: Job job_1782357914992_0005 completed successfully
2026-06-25 05:50:43,035 INFO mapreduce.Job: Counters: 54
File System Counters
  FILE: Number of bytes read=544
  FILE: Number of bytes written=554215
  FILE: Number of read operations=0
  FILE: Number of large read operations=0
  FILE: Number of write operations=0
  HDFS: Number of bytes read=422
  HDFS: Number of bytes written=386
  HDFS: Number of read operations=8
  HDFS: Number of large read operations=0
  HDFS: Number of write operations=2
  HDFS: Number of bytes read erasure-coded=0
Job Counters
  Launched map tasks=1
  Launched reduce tasks=1
  Data-local map tasks=1
  Total time spent by all maps in occupied slots (ms)=3658
  Total time spent by all reducers in occupied slots (ms)=3553
  Total time spent by all map tasks (ms)=3658
  Total time spent by all reduce tasks (ms)=3553
  Total vcore-milliseconds taken by all map tasks=3658
  Total vcore-milliseconds taken by all reduce tasks=3553
  Total megabyte-milliseconds taken by all map tasks=3745792
  Total megabyte-milliseconds taken by all reduce tasks=3638272
Map-Reduce Framework
  Map input records=5
  Map output records=40
  Map output bytes=482
  Map output materialized bytes=544
  Input split bytes=100
  Combine input records=40
  Combine output records=38
  Reduce input groups=38
  Reduce shuffle bytes=544
  Reduce input records=38
  Reduce output records=38
  Spilled Records=76
  Shuffled Maps =1
  Failed Shuffles=0
  Merged Map outputs=1
  GC time elapsed (ms)=78
  CPU time spent (ms)=1640
  Physical memory (bytes) snapshot=494055424
  Virtual memory (bytes) snapshot=5482840664
  Total committed heap usage (bytes)=392167424
  Peak Map Physical memory (bytes)=273160080
  Peak Map Virtual memory (bytes)=2736017488
  Peak Reduce Physical memory (bytes)=219705344
  Peak Reduce Virtual memory (bytes)=2746822656
Shuffle Errors
  BAD_ID=0
  CONNECTION=0
  IO_ERROR=0
  WRONG_LENGTH=0
  WRONG_MAP=0
  WRONG_REDUCE=0
File Input Format Counters
  Bytes Read=322
File Output Format Counters
  Bytes Written=386
vikas@vikash:/mnt/c/Users/Vikas$

```

Screenshot 5: Checking Output Directory

```

vikas@vikash:/mnt/c/Users/Vikas$ hdfs dfs -ls /output
2026-06-25 05:52:50,731 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
Found 2 items
-rw-r--r-- 1 vikas supergroup          0 2026-06-25 05:50 /output/_SUCCESS
-rw-r--r-- 1 vikas supergroup    386 2026-06-25 05:50 /output/part-r-000000
vikas@vikash:/mnt/c/Users/Vikas$

```

Screenshot 6: Viewing Word Count Results

```

vikas@Vikas:/mnt/c/Users/Vikas$ hdfs dfs -cat /output/part-r-00000
2026-06-25 05:53:44,792 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
Artificial 1
Big 1
Data 1
Hadoop 1
Intelligence 1
Learning 1
Machine 1
Spark 1
across 1
analytics 1
and 2
capabilities. 1
computers 1
data 1
data. 1
datasets. 1
distributed 1
enables 2
fast 1
from 1
help 1
industries 1
is 1
learn 1
learning 1
machine 1
massive 1
organizations 1
process 1
processing 1
provides 1
storage 1
technologies 1
the 1
to 1
transforming 1
workflows. 1
world. 1

```

5. Conclusion

The Hadoop MapReduce Word Count application was successfully executed. The input text file was uploaded to HDFS, processed using the built-in Hadoop Word Count program, and the output containing the frequency of each word was generated and stored in the HDFS output directory. This experiment demonstrates the basic working of Hadoop MapReduce for distributed data processing.